CAUCASUS UNIVERSITY · SCHOOL OF TECHNOLOGY

Python Programming II

Quiz (Meetings 1–5) — Option 1

Lecturer: Prof. Maksim Iavich · Time: 80 minutes · Total: 100 points

Name:		**Student ID:**	
Date:		**Score:**	/ 100

Instructions. Solve every task on the computer in Python. Save each task in its own file (task1.py … task5.py) and make sure it runs. Test your code with the examples given. Each task is worth 20 points.

Task 1 Find a word inside a string (Meeting 1 · Loops · 20 points)

Write a program that asks the user for a string s and a 3-letter word w. Using a loop, print every position (index) where w starts in s, and then the total number of times w occurs. Overlapping occurrences count. If w does not occur, print a message.

Example output:

```
Enter a string: abababxaba
Enter a 3-letter word: aba
Found at position 0
Found at position 2
Found at position 7
Total: 3

Enter a string: python
Enter a 3-letter word: bob
bob was not found
```

Task 2 Binary string → integer (Meeting 2 · Numbers · 20 points)

Write a program that asks the user for a binary number as a string (for example 10011) and converts it to a decimal integer using a loop and powers of 2. Do not use int(s, 2). If the string contains anything other than 0 and 1, print that it is not a binary number.

Example output:

```
Enter a binary number: 10011
10011 in decimal is 19

Enter a binary number: 110101
110101 in decimal is 53

Enter a binary number: 1021
1021 is not a binary number
```

Task 3 iterDiv and recurDiv (Meeting 3 · Recursion · 20 points)

Write an iterative function iterDiv(a, b) that computes the integer quotient a // b by repeatedly subtracting b from a and counting the subtractions ($a \geq 0$, $b > 0$).

Then write a recursive function recurDiv(a, b) that solves the same problem by calling itself on a smaller version of the problem. Do not use //, / or % inside the functions. Write a docstring for each function.

Example output:

```
print(iterDiv(17, 5))    ->  3
print(recurDiv(17, 5))   ->  3
print(recurDiv(20, 4))   ->  5
print(recurDiv(3, 7))    ->  0
```

Task 4 splitTuple (Meeting 4 · Tuples · 20 points)

Write a function splitTuple(aTup) that takes a tuple and returns two new tuples: the first holds the elements at even positions (0, 2, 4, …) and the second holds the elements at odd positions (1, 3, 5, …). Return both tuples at once and unpack them when calling. Use a loop, not slicing.

Example output:

```
evens, odds = splitTuple(('I', 'am', 'a', 'test', 'tuple'))
print(evens)   ->  ('I', 'a', 'tuple')
print(odds)    ->  ('am', 'test')
```

Task 5 Working with a list (Meeting 5 · Lists · 20 points)

Make it with a function:

1. Create the empty list and add 40 random integers from 1 to 50 to it.
2. Add every element divisible by 3 to a separate list.
3. Add every element whose position is divisible by 5 (positions 0, 5, 10, …) to a second separate list.
4. Print both new lists and the average of each one. Print the largest and smallest element of the main list.
5. Sort the main list in descending order, remove its 3 smallest elements with pop(), and print how many elements are left.

Example output (numbers are random and will differ):

```
Divisible by 3: [9, 42, 39, 45, 18, 15, 21, 42]
Average: 28.875
Positions 0, 5, 10, ...: [9, 8, 42, 2, 50, 47, 2, 1]
Average: 20.125
Max: 50   Min: 1
Elements left: 37
```

CAUCASUS UNIVERSITY · SCHOOL OF TECHNOLOGY

Python Programming II

Quiz (Meetings 1–5) — Option 2

Lecturer: Prof. Maksim Iavich · Time: 80 minutes · Total: 100 points

Name:		**Student ID:**	
Date:		**Score:**	/ 100

Instructions. Solve every task on the computer in Python. Save each task in its own file (task1.py … task5.py) and make sure it runs. Test your code with the examples given. Each task is worth 20 points.

Task 1 Longest substring in reverse alphabetical order (Meeting 1 · Loops · 20 points)

Write a program that prints the longest substring of s in which the letters occur in reverse alphabetical order (each letter is the same as or comes before the previous one in the alphabet). In the case of ties, print the first substring.

Example output:

```
s = 'azcbobobegghakl'
Longest substring in reverse alphabetical order is: zcb

s = 'zyxabcbbaz'
Longest substring in reverse alphabetical order is: zyxa
```

Task 2 Cube root with Newton-Raphson (Meeting 2 · Numbers · 20 points)

Write a program that asks the user for a number k and finds its cube root with the Newton-Raphson method, i.e. the root of $p(x) = x^3 - k$. Derive $p'(x)$ yourself. Start with guess = k/2 and stop when $|guess^3 - k| < 0.01$. Print the number of guesses and the result.

Example output:

```
Enter a number: 27
numGuesses = 7
Cube root of 27.0 is about 3.000000081210202

Enter a number: 64
numGuesses = 8
Cube root of 64.0 is about 4.00000913129022
```

Task 3 Recursive password check (Meeting 3 · Recursion · 20 points)

Write a recursive function count_digits(s) that returns how many digits the string s contains. Do not use loops.

Then write a function check_password(pw) that uses count_digits and prints whether the password is weak or strong. A password is weak if it is shorter than 8 characters or has fewer than 2 digits.

Example output:

```
check_password('P@ssw0rd')      ->  P@ssw0rd: 1 digit(s) - weak
check_password('abc12')         ->  abc12: 2 digit(s) - weak
check_password('Tb1lisi2026')   ->  Tb1lisi2026: 5 digit(s) - strong
```

Task 4 Least common multiple (Meeting 4 · Recursion · 20 points)

Write an iterative function lcmIter(a, b) that returns the least common multiple of two positive integers by checking multiples of the larger number until one is divisible by the smaller number.

Then write a recursive function gcdRecur(a, b) and use it in lcmRecur(a, b), using the rule lcm(a, b) = a · b / gcd(a, b). Both LCM functions must give the same result.

Example output:

```
print(lcmIter(12, 18))    ->  36
print(lcmRecur(12, 18))   ->  36
print(lcmIter(4, 6))      ->  12
print(lcmRecur(4, 6))     ->  12
```

Task 5 Functions as objects (Meeting 5 · Higher-order functions · 20 points)

1. Write a function applyNew(L, f) that returns a NEW list with f applied to every element of L, without changing L. Test it on L = [-3, 2.7, -1.2] with abs and with round, and show that L did not change.

2. Write your own function square(x). Then write applyFuns(L, x) that takes a list of functions and prints each one applied to x. Call it with [abs, round, square] and x = -5.

3. Using map, print the larger element at each position of L1 = [3, 40, 7] and L2 = [8, 12, 7].

Example output:

```
1)  [3, 2.7, 1.2]
    [-3, 3, -1]
    L is still [-3, 2.7, -1.2]
2)  5
    -5
    25
3)  8
    40
    7
```